

Technical Forensic Analysis of Navigation Mesh Incompatibilities in World of Warcraft 3.3.5a DLL Injection Scenarios

Executive Summary

The integration of autonomous navigation capabilities into legacy game client environments—specifically the World of Warcraft (WoW) 3.3.5a (Build 12340) architecture—presents a complex convergence of coordinate space mathematics, binary file structure alignment, and compile-time configuration mismatches. This report addresses the critical failure mode observed where the `findNearestPoly` function of the Recast Detour library consistently returns a null result (0) when operating on `.mmap` and `.mmtile` data generated by the TrinityCore/MaNGOS ecosystem.

The investigation has identified that the failure is not a singular runtime error but the result of a tripartite systemic incompatibility between the injected Dynamic Link Library (DLL) and the static navigation data. First, the presence of the header value `maxPolys=2^31 (2,147,483,648)` in the generated map files confirms that the data was constructed using a 64-bit Polygon Reference (DT_POLYREF64) schema. The utilizing library, however, is a standard 32-bit build (via `vcpkg`), which triggers an integer overflow during the bit-mask initialization in `dtNavMesh::init`. This overflow forces the allocation of zero bits for the salt and tile masks, rendering the mesh mathematically invalid and untraversable.

Second, the coordinate system transformation between the WoW client's proprietary Z-up world space and Recast's Y-up navigation space requires a specific permutation matrix ($\{Y, Z, X\}$) that is not natively handled by stock Detour queries. Failure to apply this swizzling transform places the query coordinates outside the bounding volume of the loaded mesh.

Third, the binary file headers generated by TrinityCore contain proprietary metadata wrappers ("MMAP" magic bytes and version identifiers) that precede the standard `dtNavMeshParams` structure. A direct binary read using standard Recast loaders results in memory misalignment, causing the parsing of garbage data as mesh parameters.

This report provides an exhaustive technical deconstruction of these failure vectors, supported by code analysis of the DetourNavMesh source, TrinityCore generation logic, and binary format specifications. It concludes with a comprehensive remediation strategy involving the custom compilation of the Recast library to match the 64-bit reference width, the implementation of a

custom binary loader to strip proprietary headers, and the precise application of coordinate transformation matrices.

1. Technological Framework and Architecture

To understand the specific failure mechanism of `findNearestPoly`, one must first establish the architectural context of the operating environment. The problem exists at the intersection of three distinct software lineages: the legacy World of Warcraft client (2010 era), the modern TrinityCore emulation codebase, and the Recast & Detour navigation middleware.

1.1 The Host Environment: World of Warcraft 3.3.5a (Build 12340)

The target application is a 32-bit x86 executable compiled for Windows. This architecture imposes strict memory addressing limits (typically 2GB or 4GB with Large Address Awareness). When a DLL is injected into this process, it shares the same linear address space as the game client. This provides the distinct advantage of low-latency access to player coordinates and object states but requires the injected code to adhere strictly to the host's memory alignment and calling conventions.

Crucially, the WoW client itself does not natively use Recast/Detour for its internal movement logic in the 3.3.5a era; it relies on a proprietary mesh and heightfield system (Collision Height Field or "CHF" and M2 collision geometry). The .mmap files in question are external artifacts, reverse-engineered and generated by the emulation community to replicate server-side pathfinding logic. This creates a fundamental "data provenance" disconnect: the client (WoW) renders the world using one set of data (ADTs/WMOs), while the injected DLL attempts to navigate using a derivative dataset (MMAPs) generated by a third-party tool (TrinityCore `mmaps_generator`). Any discrepancy between the visual world and the navigation mesh results in the agent attempting to traverse geometry that does not exist in the mesh, or vice versa.

1.2 The Navigation Middleware: Recast & Detour

Recast & Detour is the industry-standard open-source solution for navigation mesh generation and pathfinding. It operates in two distinct phases:

1. **Recast (Generation):** A compile-time or offline process that voxelizes raw geometry, filters walkable surfaces, and generates a polygonal mesh.
2. **Detour (Runtime):** A runtime library that loads the generated mesh data and performs spatial queries (pathfinding, raycasting, nearest-point lookups).

The user is employing the "Stock" version of Detour (via `vcpkg`), which implies a standard configuration intended for general-purpose development. However, the data (.mmap) was generated by TrinityCore's highly customized fork of Recast. These customizations are not

merely superficial; they alter the fundamental binary layout of the data structures.

1.3 The Data Pipeline: TrinityCore Mmap Generator

TrinityCore acts as the "author" of the binary data. The generation pipeline involves:

1. **Extraction:** mapextractor pulls raw geometry from the WoW client's MPQ archives.
2. **Assembly:** mmap4assembler constructs Line-of-Sight (LoS) models.
3. **Mesh Generation:** mmap_generator (using TerrainBuilder.cpp¹) consumes the map data and rasterizes it into Recast tiles.

The configuration settings used during step 3 are baked into the binary files. If the generator was compiled with specific preprocessor definitions—such as DT_POLYREF64—the output files carry that structural DNA. A consumer (the injected DLL) that does not share this DNA will inevitably fail to interpret the data correctly.

2. The Mathematics of Spatial Representation

A primary vector for the findNearestPoly failure is the misalignment of coordinate systems.

Navigation meshes are spatially indexed; a query is essentially a request to "find polygons

within bounding box B centered at point P ." If P is defined in Coordinate System A (WoW),

but the mesh is indexed in Coordinate System B (Recast), the query point P effectively exists in a void relative to the mesh, resulting in a return value of 0.

2.1 The World of Warcraft Coordinate Space

WoW utilizes a Right-Handed Cartesian coordinate system where **Z is Up**. This is standard for engineering and architectural applications but less common in modern graphics engines (like Unity or Unreal) which often use Y-up.

- **Origin $(0, 0, 0)$:** The center of the map (often not the gameplay center).
- **$+X$ Axis:** North.
- **$+Y$ Axis:** West (Note: In standard math, Y is often "up" on paper, but in WoW, Y is the second horizontal axis).
- **$+Z$ Axis:** Vertical elevation.

Vertices in the WoW client memory (e.g., CWorld::GetPlayerPos) are stored as an array of three floats: ``.

2.2 The Recast Coordinate Space

Recast assumes a Right-Handed coordinate system where **Y is Up**. This aligns with OpenGL conventions.

- **+X Axis:** Horizontal.
- **+Y Axis:** Vertical elevation.
- **+Z Axis:** Depth.

When TrinityCore generates maps, it performs a "swizzle" operation to map the WoW world into the Recast world. This transformation is necessary because Recast's internal algorithms (e.g., rcRasterizeTriangles) assume the heightfield is built along the Y-axis.

2.3 The Coordinate Transformation Matrix

The user noted that the coordinates in the .mmap file appear to be {Y, Z, X}. This observation is critical. It implies a specific permutation of the axes. Let us analyze the transformation mathematically.

Let $V_{wow} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ be a vertex in WoW space.

Let $V_{recast} = \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$ be the corresponding vertex in the navigation mesh.

The standard TrinityCore conversion logic found in TerrainBuilder.cpp typically performs the following mapping to align the coordinate systems:

1. **WoW X** becomes **Recast Z** (swapping North/South to Depth).
2. **WoW Y** becomes **Recast X** (swapping West/East to Width).
3. **WoW Z** becomes **Recast Y** (swapping Elevation to Height).

However, chirality (handedness) must be preserved. A simple swap might invert the world. The transformation observed {Y, Z, X} implies:

$$x' = -y_{wow}$$

$$y' = z_{wow}$$

$$z' = -x_{wow}$$

(Note: The negation signs depend on the specific origin offset used by the tile generator.

TrinityCore creates tiles based on a global grid where the origin is often the top-left corner of the world, requiring axis inversion).

The Failure Mechanism:

If the injected DLL reads the player's position from memory as $P_{player} = (1000, 500, 25)$ (where 25 is height), and passes this directly to findNearestPoly:

- Detour interprets this as: $X = 1000, Y = 500(\text{height}), Z = 25$.
- The actual mesh geometry at that location stores the height in the Y channel.
- The query searches for a polygon at height 500. The actual terrain is at height 25 (stored as Y in the mesh).
- Since the vertical difference ($500 - 25 = 475$) likely exceeds the search extents (usually $\{2, 4, 2\}$), the query fails.

2.4 Extents and The Search Box

The findNearestPoly function takes a halfExtents vector, which defines the search radius.

C++

```
dtNavMeshQuery::findNearestPoly(const float* center, const float* halfExtents,...);
```

If halfExtents is set to $\{1.0f, 1.0f, 1.0f\}$, the function searches a box of size $2 \times 2 \times 2$.

If the coordinate systems are mismatched, the Euclidean distance between the query point and the nearest polygon can be thousands of units. No reasonable extent value can bridge this gap. The coordinate systems **must** be aligned via a pre-query transformation step.

3. The DT_POLYREF64 Incompatibility: The Core Schism

The most definitive evidence of incompatibility provided in the query is the header value

$\text{maxPolys} = 2^{31}$ ($2,147,483,648$). This value is physically impossible for a standard 32-bit Recast build to handle and is the primary reason for the "silent failure" of the navigation mesh

initialization.

3.1 The Theory of Polygon References (dtPolyRef)

In Detour, a dtPolyRef is not a pointer. It is a packed integer handle that uniquely identifies a polygon across the entire mesh. It allows the system to validate references even if tiles are unloaded or reloaded. The handle packs three distinct pieces of information into a single integer:

1. **Salt** (S): A counter incremented every time a tile is reloaded. This invalidates old references held by agents (e.g., if a door opens/closes and the tile rebuilds).
2. **Tile Index** (T): The index of the tile in the mesh array.
3. **Poly Index** (P): The index of the polygon within that specific tile.

The allocation of bits for S , T , and P is determined dynamically at runtime during dtNavMesh::init, based on the maxPolys and maxTiles parameters passed in the dtNavMeshParams structure.

3.2 32-bit Reference Architecture (Stock Recast)

In the default vcpkg distribution, DT_POLYREF64 is undefined. The dtPolyRef type is unsigned int (32 bits).

The logic in DetourNavMesh.cpp calculates the bit widths as follows:

Let $W = 32$ (Total width).

1. Calculate bits required for Tile Index: $B_{tile} = \lceil \log_2(\text{params.maxTiles}) \rceil$.
2. Calculate bits required for Poly Index: $B_{poly} = \lceil \log_2(\text{params.maxPolys}) \rceil$.
3. Calculate remaining bits for Salt: $B_{salt} = W - B_{tile} - B_{poly}$.

The Constraint:

Recast imposes a hard limit to ensure safety:

C++

```
if (m_saltBits < 10) return DT_FAILURE;
```

This requires that at least 10 bits be reserved for the salt, leaving only 22 bits for Tiles + Polys combined.

3.3 The Integer Overflow Scenario

The user's .mmap file specifies $\text{maxPolys} = 2^{31}$.

- Input $\text{params.maxPolys} = 2,147,483,648$.
- Calculation: $B_{\text{poly}} = \lceil \log_2(2^{31}) \rceil = 31$.

If the system attempts to initialize with this value on a 32-bit build:

- $B_{\text{poly}} = 31$ bits.
- Even if maxTiles is 1 ($B_{\text{tile}} = 1$ bit):
- $B_{\text{salt}} = 32 - 31 - 1 = 0$ bits.

Result: The calculation results in 0 bits for salt. The `dtNavMesh::init` function will either return `DT_FAILURE` (if the check is respected) or, if the check is bypassed or the overflow wraps around in a specific way, it will initialize a corrupt mesh state where the bitmasks are invalid.

Specifically, if `m_tileBits` or `m_polyBits` consumes the entire integer width, the bitwise shift operations used to extract indices:

C++

```
unsigned int tileIndex = (ref >> m_polyBits) & m_tileMask;
```

will behave unpredictably (undefined behavior in C++ to shift by $\geq$ width of type), often resulting in 0.

3.4 64-bit Reference Architecture (DT_POLYREF64)

TrinityCore defines `DT_POLYREF64` in its CMake configuration. This changes `dtPolyRef` to `uint64_t`.

- $W = 64$.
- $B_{\text{poly}} = 31$ (derived from 2^{31} header).

- B_{tile} is typically around 16 bits (allowing 65,535 tiles).
- Remaining bits for Salt: $64 - 31 - 16 = 17$ bits.

This is a valid configuration. The value 2^{31} in the header is an explicit signal that the file was generated for a 64-bit reference system. By attempting to load this into a 32-bit Detour library, the user is forcing a square peg into a round hole. The library cannot address the polygons because it physically lacks the bits to represent the indices implied by the header.

4. Binary File Forensics and Struct Alignment

Beyond the logical incompatibility of the bit-packing, there is a binary layout incompatibility. The C++ language does not guarantee binary compatibility between different compiler settings, particularly when fundamental types change size.

4.1 The .mmap Container Format

TrinityCore does not output raw memory dumps of Recast structs. It uses a custom container format for .mmap files to handle versioning and map identification.

Structure of a TrinityCore .mmap file:

Offset (Bytes)	Field	Type	Value	Notes
0x00	Magic	char	"MMAP"	Custom TC Identifier
0x04	Version	uint32	4, 5, or 6	Depends on TC revision
0x08	MapID	uint32	e.g., 530	WoW Map ID
0x0C	dtNavMeshParams	Struct	varies	The actual Recast data starts here

The Loading Error:

Standard Recast code typically assumes the file starts immediately with dtNavMeshParams or

dtMeshHeader. If the user's code calls:

```
C++
```

```
fread(&params, sizeof(dtNavMeshParams), 1, fp);
```

It will read the first 12 bytes ("MMAP" + Version + MapID) into the first three fields of dtNavMeshParams (origin, origin, origin).

- origin becomes the float representation of "MMAP".
- maxPolys (field 5) will read garbage data from further down the file, or potentially the actual origin data depending on struct packing.

However, the user *successfully* identified maxPolys= 2^{31} . This implies they might be skipping the header correctly, or the garbage data coincidentally aligns. Given the precision of the value (2^{31} is exactly 0x80000000), it is highly likely the user is parsing the file correctly up to the dtNavMeshParams, but failing at the implementation of init.

4.2 The .mmtile Tile Header Discrepancy

The mesh geometry resides in .mmtile files. These also contain a TrinityCore-specific header before the Recast data.

```
C++
```

```
struct MmapTileHeader {  
    uint32 size;  
    uint32 balance; // Checksum or padding  
};
```

More critically, the actual Recast tile data begins with dtMeshHeader.

Structure of dtMeshHeader:

```
C++
```

```

struct dtMeshHeader {
    int magic;
    int version;
    int x;
    int y;
    int layer;
    unsigned int userId; // <--- CRITICAL FAILURE POINT
    int polyCount;
    int vertCount;
    //...
};

```

Impact of DT_POLYREF64 on Alignment:

The field `userId` is of type `dtPolyRef`.

- **Stock Recast:** `dtPolyRef` is 4 bytes (`uint32`).
- **TrinityCore:** `dtPolyRef` is 8 bytes (`uint64_t`).

When TrinityCore writes the tile, it writes 8 bytes for `userId`.

When the Stock DLL reads the tile, it reads 4 bytes for `userId`.

Consequence: Every field after `userId` is shifted by 4 bytes.

- `polyCount` reads the lower half of the 64-bit `userId` (or the upper half depending on endianness).
- `vertCount` reads the old `polyCount`.
- The `bvTree` pointer offsets are read as garbage.

When `findNearestPoly` attempts to traverse the `BVTree` using these corrupted offsets, it accesses invalid memory. In a robust system, this crashes. In a release build with relaxed checks, it simply fails to find a leaf node and returns 0.

5. Algorithmic Breakdown of `findNearestPoly` Failure

To understand exactly *where* the function returns 0, we trace the execution path of `dtNavMeshQuery::findNearestPoly` given the identified incompatibilities.

5.1 The Algorithm Steps

1. **Coordinate Quantization:** The query center is verified against the mesh bounds. (Fails if coordinates are not swizzled).

2. **Tile Lookup:** `m_nav->getTileAt(x, y, layer)` is called to find the tile at the query location.
 - This relies on `m_posLookup` bucket array.
 - If init failed (due to the bit-width issue), the lookup table may be empty or uninitialized.
3. **Broadphase Search (BVTree):** If a tile is found, the algorithm traverses the tile's Bounding Volume Tree (`bvTree`) to find polygons that overlap the query extents.
 - The BVTree node indices are stored relative to the tile data.
 - Due to the **4-byte alignment shift** in `dtMeshHeader` caused by the `dtPolyRef` size mismatch, the values in `header->bvNodeCount` and the root node index are corrupt.
 - The traversal loop `while (node < end)` will likely terminate immediately (if `nodeCount` reads as 0) or fail intersect tests (if nodes read as garbage floats).
4. **Polygon Refinement:** The algorithm checks the distance to polygons found.
 - If the BVTree returns no candidates (due to corruption), the function returns 0.

5.2 The "Silent" Failure

The function `findNearestPoly` is designed to be robust; if it finds no polygons, it returns a status of `DT_SUCCESS` but sets the output `nearestRef` to 0. This explains why the user sees a "success" return code (or lack of crash) but gets no result. The function is working correctly; the data it is processing describes an empty or invalid void.

6. Implementation of the Solution

The incompatibility is structural and cannot be resolved by changing runtime parameters alone. The injected DLL **must** be recompiled to match the binary footprint of the TrinityCore generator.

6.1 Step 1: Forking and Compiling Recast

The user must uninstall the `vcpkg` version of Recast. It is unsuitable for this task because `vcpkg` aims for standard ABI compatibility, whereas TrinityCore breaks standard ABI.

Action:

1. Clone the specific version of Recast used by the TrinityCore build (usually found in `dep/recastnavigation` of the TC source tree). This ensures protocol version matches.
2. Configure the build system (CMake) to enable 64-bit polygon references.

CMake Instruction:

CMake

```
# In the DLL's CMakeLists.txt
add_definitions(-DDT_POLYREF64)
```

Verification:

Add a static assertion in the DLL source to guarantee the type size:

C++

```
static_assert(sizeof(dtPolyRef) == 8, "Recast must be compiled with 64-bit refs to match TrinityCore maps");
```

6.2 Step 2: Implementation of the Custom Loader

A generic dtNavMesh::load call will fail due to the proprietary headers. A custom loader must be implemented to strip the TC metadata.

Code Example: Custom TC.mmap Loader

C++

```
#include "DetourNavMesh.h"
#include "DetourCommon.h"
#include <cstdio>
#include <stdint>

// TrinityCore.mmap header structure
struct TCMmapHeader {
    char magic; // "MMAP"
    uint32_t version; // TC Version
    uint32_t mapID; // WoW Map ID
};

dtNavMesh* LoadTrinityNavMesh(const char* filepath) {
    FILE* fp = fopen(filepath, "rb");
    if (!fp) return nullptr;
```

```

// 1. Read and Skip TC Header
TCMmapHeader header;
size_t read = fread(&header, sizeof(TCMmapHeader), 1, fp);
if (read != 1 |

| strcmp(header.magic, "MMAP", 4) != 0) {
    // Log Error: Invalid Magic
    fclose(fp);
    return nullptr;
}

// 2. Read dtNavMeshParams
dtNavMeshParams params;
read = fread(&params, sizeof(dtNavMeshParams), 1, fp);
if (read != 1) { fclose(fp); return nullptr; }

// 3. Allocate Mesh
dtNavMesh* mesh = dtAllocNavMesh();
if (!mesh) { fclose(fp); return nullptr; }

// 4. Initialize (Must have DT_POLYREF64 defined!)
dtStatus status = mesh->init(&params);
if (dtStatusFailed(status)) {
    // Log Error: Initialization failed. Likely bit-width mismatch if maxPolys is huge.
    dtFreeNavMesh(mesh);
    fclose(fp);
    return nullptr;
}

fclose(fp);
return mesh;
}

```

6.3 Step 3: Loading Tiles with Header Skipping

Similarly, tiles must be loaded by skipping the MmapTileHeader.

C++

```

struct TCMtileHeader {

```

```

uint32_t size;
uint32_t balance;
};

bool LoadTrinityTile(dtNavMesh* mesh, const char* filepath) {
    FILE* fp = fopen(filepath, "rb");
    if (!fp) return false;

    TCMtileHeader header;
    fread(&header, sizeof(TCMtileHeader), 1, fp);

    // Alloc buffer for the actual Recast data
    unsigned char* data = (unsigned char*)dtAlloc(header.size, DT_ALLOC_PERM);
    fread(data, header.size, 1, fp);
    fclose(fp);

    // addTile takes ownership of the pointer
    dtStatus status = mesh->addTile(data, header.size, DT_TILE_FREE_DATA, 0, nullptr);
    return dtStatusSucceed(status);
}

```

6.4 Step 4: Runtime Coordinate Transformation

The injected DLL must intercept the player's position and transform it *before* querying the mesh.

Transformation Logic:

C++

```

// Input: Player Position from WoW Memory (Z-up)
float wowPos = { player.x, player.y, player.z };

// Output: Search Position for Recast (Y-up)
// Based on {Y, Z, X} observation
float searchPos;
searchPos = -wowPos; // Recast X = -WoW Y
searchPos = wowPos; // Recast Y = WoW Z
searchPos = -wowPos; // Recast Z = -WoW X

// Search Extents (Y is height in Recast)

```

```
float extents = { 2.0f, 4.0f, 2.0f };
```

```
dtPolyRef ref;
```

```
float nearestPt;
```

```
query->findNearestPoly(searchPos, extents, &filter, &ref, nearestPt);
```

Note on Negation: The negation of X and Y axes depends on the specific grid origin used by TerrainBuilder. If the mesh appears loaded but points are still not found, invert the negation (e.g., try searchPos = wowPos). The swizzle {-Y, Z, -X} is the most common standard for converting WoW's "North=X" system to Recast's standard orientation.

7. Broader Implications for Emulation Development

This failure case highlights a significant systemic risk in game emulation development: the **Implicit Interface Dependency**. While the .mmap file extension suggests a standard format, the file is actually a versioned, compiler-dependent binary snapshot of a C++ object.

7.1 The Fragility of ABI

The fact that dtPolyRef size changes the offset of fields in dtMeshHeader is a violation of robust binary file design, yet it is common in high-performance middleware where serialization speed is prioritized over compatibility. By dumping structs directly to disk, TrinityCore creates a hard dependency on the exact compiler flags (padding, packing, type sizes) used at generation time.

7.2 Versioning "Hell"

TrinityCore updates its map format frequently (v4, v5, v6, etc.). The user is on Build 12340 (3.3.5a), which corresponds to a specific era of TrinityCore (circa 2010-2012, or modern "3.3.5 branch"). Using a modern Recast library (vcpkg typically pulls the latest HEAD) against 2010-era data structures invites subtle version mismatches beyond just the 64-bit reference issue. It is strongly recommended to check the MMAP version in the file header and cross-reference it with the TrinityCore source code history to ensure the dtNavMeshParams struct definition hasn't changed in other ways (e.g., addition of new fields like maxVertsPerPoly).

8. Conclusion

The inability of the user's DLL to find polygons using findNearestPoly is a deterministic consequence of loading 64-bit, coordinate-swizzled, proprietary-header-wrapped data into a stock 32-bit library. The data is effectively "invisible" to the library because:

1. **Bit-width mismatch** prevents the mesh from indexing tiles (Salt/Tile bits = 0).
2. **Alignment mismatch** corrupts the geometric data in the tiles (User ID shift).
3. **Coordinate mismatch** places the query in the wrong spatial octant.

The remediation requires a full recompilation of the navigation stack with DT_POLYREF64 enabled and the implementation of a proprietary loading layer to handle the TrinityCore file wrappers. Without these binary-level adjustments, the navigation mesh will remain opaque to the injected logic.

End of Report.

Works cited

1. accessed January 1, 1970,
https://github.com/TrinityCore/TrinityCore/blob/3.3.5/src/tools/mmap_generator/TerrainBuilder.cpp
2. TrinityCore/src/tools/mmaps_generator/MapBuilder.cpp at master - GitHub,
accessed February 19, 2026,
https://github.com/TrinityCore/TrinityCore/blob/master/src/tools/mmaps_generator/MapBuilder.cpp